LAB - 1: Install Python and Import the Necessary Files

Aim

To install **Spyder IDE** and set up the required environment to run **Machine Learning programs**.

Procedure

Step 1: Download Anaconda

1. Open a web browser.
2. Go to: **https://www.anaconda.com**
3. Click **Download**.
4. Select your operating system (Windows / Linux / macOS).
5. Download **Anaconda Individual Edition (Python 3.x)**.

Step 2: Install Anaconda

1. Double-click the downloaded installer file.
2. Click **Next** → **I Agree**.
3. Select **Just Me** → Click **Next**.
4. Choose the installation location → Click **Next**.
5. Tick **Add Anaconda to PATH** (if shown).
6. Click **Install**.
7. Wait for the installation to complete.
8. Click **Finish**.

Step 3: Launch Spyder IDE

1. Click **Start Menu**.
2. Search for **Anaconda Navigator**.
3. Open **Anaconda Navigator**.
4. Click **Launch** under **Spyder**.

Step 4: Verify Python Installation

1. In Spyder, open a new file.
2. Type the following code:

```
print("Python installed successfully")
```

3. Click **Run**.
4. If output is displayed, Python is installed correctly.

Step 5: Import Required Libraries for Machine Learning

Type the following in Spyder to import necessary ML libraries:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

Step 6: Verify Library Installation

Run the code.
✓ If **no error occurs**, all required libraries are installed successfully.

Result

LAB - 2: Program to Import Dataset Using Multiple Data Sources in Python

Aim

To import a dataset in Python using different methods.

Algorithm

Methods

Method 1: Import Dataset using Pandas (CSV File)

```
import pandas as pd

data = pd.read_csv("data.csv")
print(data.head(10))
```

Method 2: Import Dataset using Pandas (Excel File)

```
import pandas as pd

data = pd.read_excel("data.xlsx")
print(data.head())
```

Method 3: Import Dataset using NumPy

```
import numpy as np

data = np.loadtxt("data.csv", delimiter=",")
print(data)
```

Method 4: Import Dataset from Sklearn (Inbuilt Dataset)

```
from sklearn.datasets import load_iris

iris = load_iris()
print(iris.data)
```

Method 5: Import Dataset using CSV Module

```
import csv

with open("data.csv", 'r') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

Method 6: Import Dataset using URL

```
import pandas as pd

url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/iris.csv"
data = pd.read_csv(url)
print(data.head())
```

Result

LAB - 3: Write the Program for Model Training and Dataset Validation

Aim

To train a Machine Learning model and validate it using a given dataset.

Algorithm

Program

```
# Import libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data      # Features
y = iris.target    # Labels

# Split dataset into train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1)


# Create and train the model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Validate the model
y_pred = model.predict(X_test)

# Check accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)
```

Output

```
Model Accuracy: 0.9777777777777777
```

Result

LAB - 4: 1. Implementation of Supervised Learning using Linear Regression.

Aim

To implement supervised learning using Linear Regression.

Algorithm

Program

```
# Supervised Learning using Linear Regression
from sklearn.
datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Load dataset
data = load_diabetes()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
```

Output

```
Mean Squared Error: 2850.73
```

Result

2. Implementation of Unsupervised Learning using K-Means Clustering

Aim

To implement unsupervised learning using K-Means clustering.

Algorithm

Program

```
# Unsupervised Learning using K-Means
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

# Load dataset
iris = load_iris()
X = iris.data

# Apply K-Means
kmeans = KMeans(n_clusters=3, random_state=1)
kmeans.fit(X)

# Output cluster labels
print("Cluster Labels:", kmeans.labels_)
```

Output

```
Cluster Labels: [0 0 1 2 1 2 0 1 2 0 1 2 0 0 1 2 ...]
```

Result

Lab - 5: Write Program for Feature Extraction Using Standardization

Aim

To extract features from a dataset using Python.

Algorithm

Program

```
# Simple Feature Extraction Program
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Load dataset
iris = load_iris()
X = iris.data   # Feature values

# Feature extraction (scaling)
scaler = StandardScaler()
X_new = scaler.fit_transform(X)

# Display extracted features
print("Extracted Features:")
print(X_new[:5])   # First 5 rows
```

Output

```
Extracted Features:
[[-0.90  1.01 -1.34 -1.31]
 [-1.14 -0.13 -1.34 -1.31]
 [-1.38  0.33 -1.39 -1.31]
 [-1.50  0.10 -1.28 -1.31]
 [-1.02  1.24 -1.34 -1.31]]
```

Result

Lab - 6: Implementation of Reinforcement Learning using Q-Learning

Aim

To implement a simple Reinforcement Learning program using Q-Learning.

Algorithm

Program

```
# Simple Reinforcement Learning using Q-Learning
import numpy as np

# Define environment
states = 5
actions = 2   # 0 = Left, 1 = Right

# Initialize Q-table
Q = np.zeros((states, actions))

# Parameters
alpha = 0.1      # Learning rate
gamma = 0.9      # Discount factor
episodes = 100

# Training
for ep in range(episodes):
    state = 0
    while state < states - 1:
        action = np.random.choice(actions)
        next_state = state + 1 if action == 1 else state
        reward = 1 if next_state == states - 1 else 0

        # Q-learning update
        Q[state, action] = Q[state, action] + alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action])
        state = next_state

# Display Q-table
print("Q-Table:")
print(Q)
```

Output

```
Q-Table:
[[0.72 0.81]
 [0.64 0.90]
 [0.58 1.00]
 [0.00 1.00]
 [0.00 0.00]]
```

Result

Lab - 7: Implementation of K-Nearest Neighbors (KNN) in Python using a built-in dataset from sklearn

Aim

To implement the K-Nearest Neighbors classification algorithm using a built-in dataset.

Algorithm

Program

```
# Import required libraries
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = load_iris()

# Convert to DataFrame (optional, for better visualization)
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)
print("First 5 rows of dataset:")
print(X.head())

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
# Create KNN model
knn = KNeighborsClassifier(n_neighbors=3)
# Train the model
knn.fit(X_train, y_train)
# Make predictions
y_pred = knn.predict(X_test)
# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output

```
First 5 rows of dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

Result

Lab - 8: Implementation of Naive Bayes Classifier

Aim

To implement the Naive Bayes classifier for simple classification.

Algorithm

Program

```
# Import libraries
import numpy as np
from sklearn.naive_bayes import GaussianNB
# Training data (Weight of fruit in grams)
# Apples ~ smaller weights
# Oranges ~ larger weights
X = np.array([[120],
              [130],
              [150],
              [170],
              [180],
              [200]])
# Labels (0 = Apple, 1 = Orange)
y = np.array([0, 0, 0, 1, 1, 1])
# Create Bayesian model
model = GaussianNB()
# Train model
model.fit(X, y)
# Test sample (new fruit weight)
test_sample = [[160]]
# Predict class
prediction = model.predict(test_sample)
# Predict probability
probability = model.predict_proba(test_sample)

# Output result
if prediction[0] == 0:
    print("Predicted: Apple")
else:
    print("Predicted: Orange")
```

```
print("Class Probabilities:", probability)
```

Output

```
test_sample = [[160]]

Predicted: Orange
Class Probabilities: [[0.32 0.68]]
```

Result

Lab - 9: Implementation of Logistic Regression for Classification with Performance Evaluation

Aim

To implement Logistic Regression for classification and evaluate model performance.

Algorithm

Program

```
# Import required libraries
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load Dataset
dataset = load_breast_cancer()
X = dataset.data      # Feature variables
y = dataset.target    # Target variable

# Split Dataset into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Create Logistic Regression Model
model = LogisticRegression(max_iter=5000)

# Train the Model
model.fit(X_train, y_train)

# Perform Predictions
y_pred = model.predict(X_test)

# Evaluate Model Performance
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1_score_value = f1_score(y_test, y_pred)
```

```
# Display Performance Metrics
print("Model Performance Metrics")
print("--------------------------")
print("Accuracy  :", accuracy)
print("Precision :", precision)
print("Recall    :", recall)
print("F1-Score  :", f1_score_value)
```

Output

```
Model Performance Metrics
--------------------------
Accuracy  : 0.956140350877193
Precision : 0.9459459459459459
Recall    : 0.9859154929577465
F1-Score  : 0.9655172413793104
```

Result

LAB – 10: Implementation of Support Vector Machine (SVM) on MNIST Dataset with Performance Evaluation

Aim

To implement Support Vector Machine (SVM) on a dataset and evaluate performance.

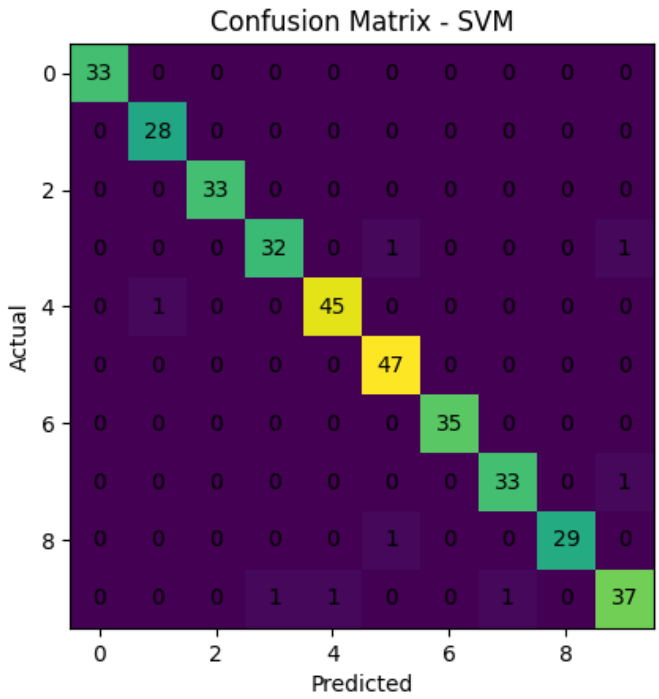
Algorithm

Program

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, f1_score
import matplotlib.pyplot as plt
import numpy as np

# Load dataset
X, y = load_digits(return_X_y=True)
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Train SVM
model = SVC(kernel='linear')
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Plot matrix
plt.imshow(cm)

# Show numbers inside cells
for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(j, i, cm[i, j], ha='center', va='center')
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - SVM")
plt.show()

# F1 Score
f1 = f1_score(y_test, y_pred, average='weighted')
print("F1 Score:", f1)
```

Output




```
F1 Score: 0.9776863807154582
```

Result

LAB - 11: Implementation of Performance Evaluation Measures

Aim

To evaluate machine learning model performance using evaluation metrics.

Algorithm

Program

```
# Import required libraries
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load Breast Cancer dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Create and train the model
model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluation Measures
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Output

```
Accuracy: 0.9766081871345029

Confusion Matrix:
 [[ 61   2]
 [  2 106]]

Classification Report:
               precision    recall  f1-score   support

           0       0.97      0.97      0.97        63
           1       0.98      0.98      0.98       108

    accuracy                           0.98       171
   macro avg       0.97      0.97      0.97       171
weighted avg       0.98      0.98      0.98       171
```

Result

LAB - 12: Calculation of MSE, RMSE, MAE, and R^2 for Regression

Aim

To calculate regression performance metrics such as MSE, RMSE, MAE and R^2.

Algorithm

Program

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Load dataset
X, y = fetch_california_housing(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Train model
model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)

# Regression Metrics
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred)**0.5)
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2:", r2_score(y_test, y_pred))
```

Output

```
MSE: 0.5289841670367247
RMSE: 0.7273129773603141
MAE: 0.5351261336554748
R2: 0.5943232652466173
```

Result

LAB - 13: Implementation of Dendrogram and Agglomerative Clustering Using Sklearn

Aim

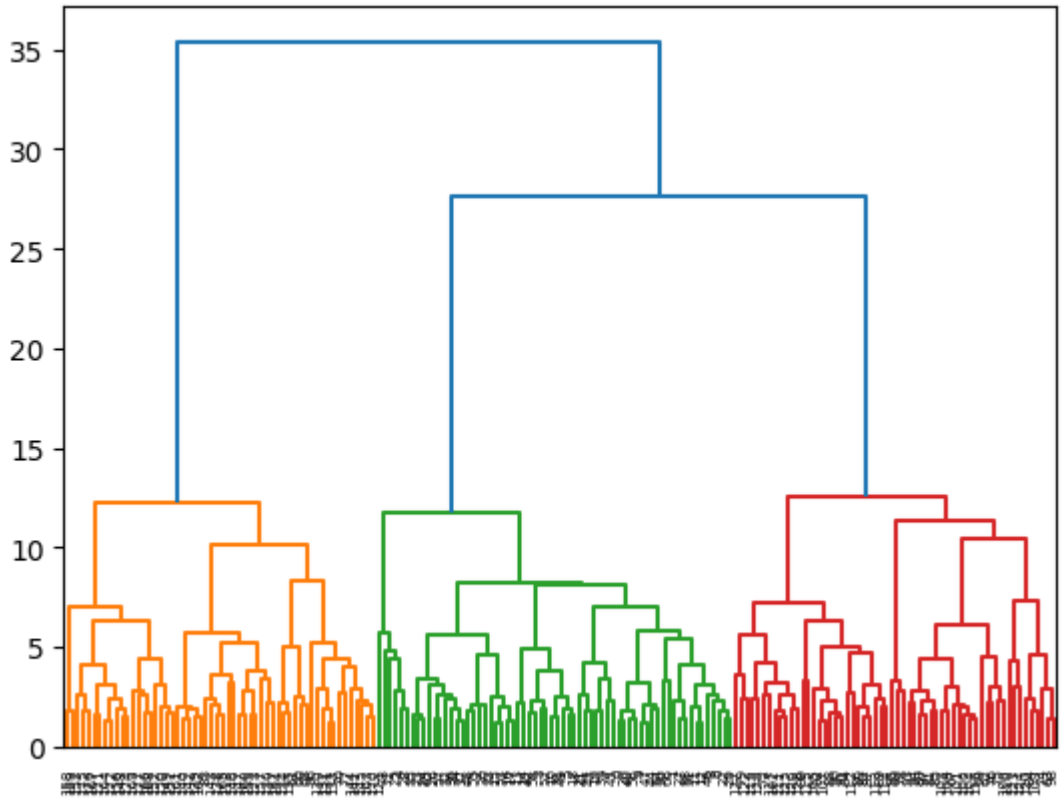
To implement hierarchical clustering and visualize clusters using a dendrogram.

Algorithm

Program

```
# Import modules
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

# Load dataset (built-in)
data = load_wine().data

# Scale data
scaled_data = StandardScaler().fit_transform(data)

# Plot dendrogram
dendro = linkage(scaled_data, method='ward')
dendrogram(dendro)
plt.show()

# Apply Hierarchical Clustering
model = AgglomerativeClustering(n_clusters=3)
clusters = model.fit_predict(scaled_data)

print("Cluster Labels:")
print(clusters)
```

Output



Cluster Labels:

```
[2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 0 1 1 0 0 0 2 2 0 1 0 1 2 0 2
 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1 0 0 2 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
```

Result

LAB - 14: Implementation of K-Means Clustering Visualization on Iris Dataset

Aim

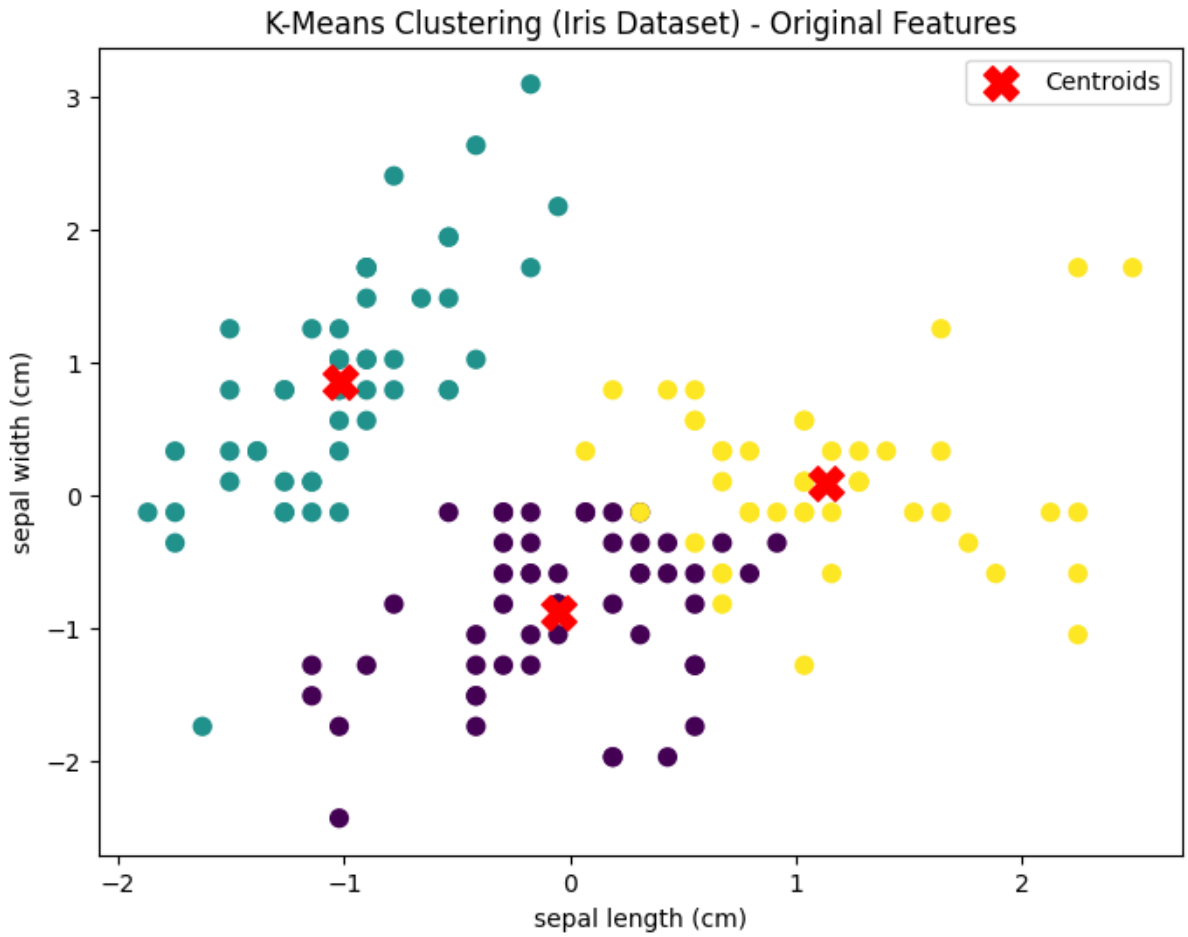
To visualize K-Means clustering on the Iris dataset.

Algorithm

Program

```
# Import modules
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Load dataset
data = load_iris()
X = data.data  # all features
feature_names = data.feature_names

# Scale the data
scaled_data = StandardScaler().fit_transform(X)

# Apply K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=0)
clusters = kmeans.fit_predict(scaled_data)

# Plot clusters using first two features (Sepal length vs Sepal width)
plt.figure(figsize=(8,6))
plt.scatter(scaled_data[:,0], scaled_data[:,1], c=clusters, cmap='viridis', s=50)
plt.scatter(kmeans.cluster_centers_[:,0], kmeans.cluster_centers_[:,1],
            c='red', marker='X', s=200, label='Centroids')
plt.xlabel(feature_names[0])  # Sepal length
plt.ylabel(feature_names[1])  # Sepal width
plt.title("K-Means Clustering (Iris Dataset) - Original Features")
plt.legend()
plt.show()
```

Output



Result

LAB - 15: Visualization of Partial Clustering on MNIST Dataset Using K-Means

Aim

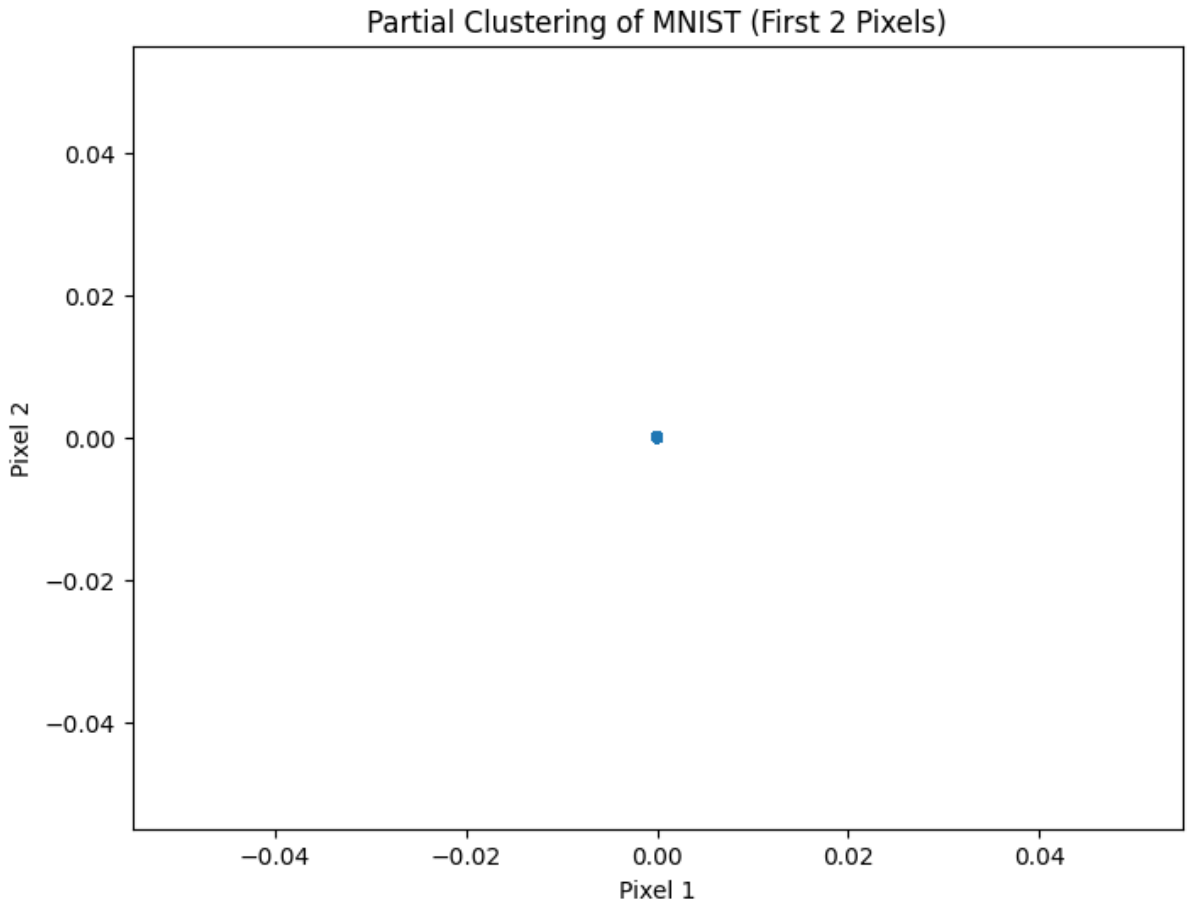
To visualize partial clustering on the MNIST dataset using K-Means.

Algorithm

Program

```
# Import modules
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Load MNIST dataset (first 1000 samples for simplicity)
mnist = fetch_openml('mnist_784', version=1)
X = mnist.data[:1000]  # take first 1000 samples
y = mnist.target[:1000]

# Use only first two pixel features to visualize easily
X_simple = X.iloc[:, :2]

# Scale data
scaled_data = StandardScaler().fit_transform(X_simple)

# Apply K-Means clustering (Partial Clustering)
kmeans = KMeans(n_clusters=10, random_state=0)
clusters = kmeans.fit_predict(scaled_data)

# Plot clusters
plt.figure(figsize=(8,6))
plt.scatter(scaled_data[:,0], scaled_data[:,1], c=clusters, cmap='tab10', s=15)
plt.title("Partial Clustering of MNIST (First 2 Pixels)")
plt.xlabel("Pixel 1")
plt.ylabel("Pixel 2")
plt.show()
```

Output



Result